

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

October 23, 2025

Lecture 17

1. What Are the Different Types of Machine Learning?

2. What is Overfitting, and How Can You Avoid It?

28. How to identify whether the model has overfitted the training data or not?

29. What is regularization, why do we use it, and give some examples of common methods

3. What is 'training Set' and 'test Set' in a Machine Learning Model? How Much Data Will You Allocate for Your Training, Validation, and Test Sets?

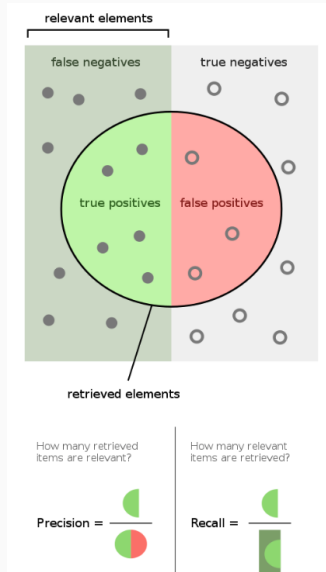
21. What is Cross-Validation?

38. What are dropouts

5. How Can You Choose a Classifier Based on a Training Set Data Size?

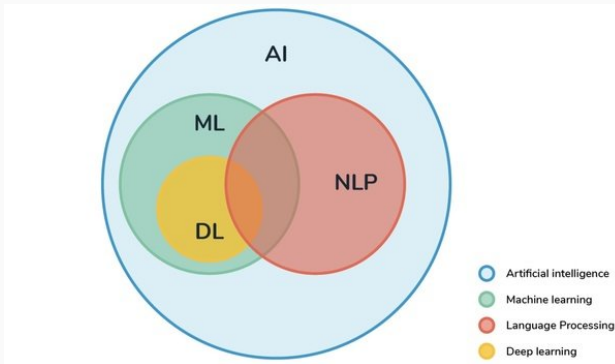
7. What Is a False Positive and False Negative and How Are They Significant?

7. What Is a False Positive and False Negative and How Are They Significant?



13. Define Precision and Recall. 33. Define F1-score: $\frac{2PR}{P+R}$

8. What Are the Differences Between Machine Learning and Deep Learning?



(Figure from "https://www.researchgate.net/figure/Relationship-between-AI-ML-DL-and-NLP-7_fig8_343079524")

16. Briefly Explain Logistic Regression. 48. Linear Regression?

9. Compare K-means and KNN Algorithms. 17. Explain the K Nearest Neighbor Algorithm.

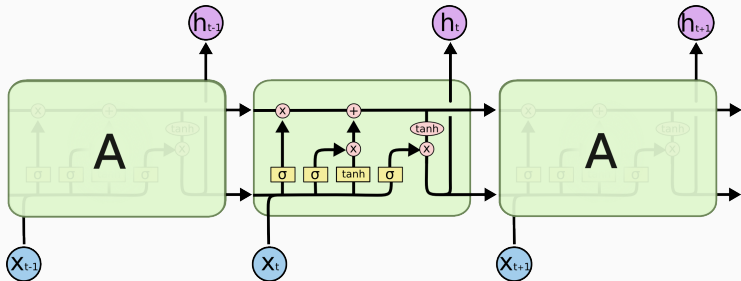
27. What is the difference between stochastic gradient descent (SGD) and gradient descent (GD)?

22. What is the difference between Lasso and Ridge regression?

34. What is cost function?

35. List different activation neurons or functions

39. Define LSTM



$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o)$$

$$\tilde{c}_t = \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \sigma_h(c_t)$$

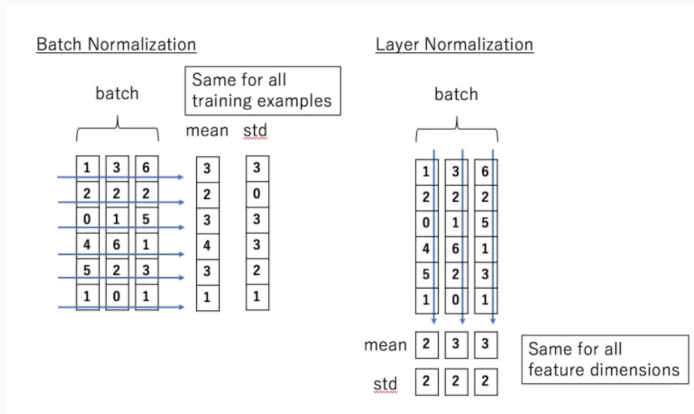
46. What's MLE/MAP? 47. Bayesian vs Frequency

Bayesian Rule

$$\underbrace{P(W|X)}_{\text{Posterior}} = \frac{\overbrace{P(X|W)}^{\text{Likelihood}} * \overbrace{P(W)}^{\text{Prior}}}{P(X)}$$
$$\propto P(X|W) * P(W)$$

50. What is batch normalization and why does it work? 30.
What is layer normalization and why does it work?

LayerNorm BatchNorm



https://medium.com/@florian_algo/batchnorm-and-layernorm-2637f46a998b

LayerNorm BatchNorm

```
CLASS torch.nn.LayerNorm(normalized_shape, eps=1e-05, elementwise_affine=True, bias=True, device=None, dtype=None) [SOURCE]
```

Applies Layer Normalization over a mini-batch of inputs.

This layer implements the operation as described in the paper [Layer Normalization](#)

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

The mean and standard-deviation are calculated over the last D dimensions, where D is the dimension of `normalized_shape`. For example, if `normalized_shape` is `(3, 5)` (a 2-dimensional shape), the mean and standard-deviation are computed over the last 2 dimensions of the input (i.e. `input.mean((-2, -1))`). γ and β are learnable affine transform parameters of `normalized_shape` if `elementwise_affine` is `True`. The standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`.

**What are the different tasks in NLP? What's their evaluations
(x10 points question)**

What are the different tasks in NLP? What's their evaluations
What are the different types of parsing in NLP?

What do you mean by vector space in NLP? What are word embeddings in NLP?

What is stemming in NLP, and how is it different from lemmatization?

What do you mean by Sequence in the Context of NLP?

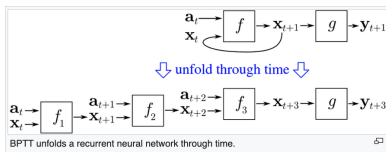
How does the Backpropagation through time work in RNN?

How does the Backpropagation through time work in RNN?

Backpropagation through time (BPTT) is a gradient-based technique for training certain types of recurrent neural networks. It can be used to train Elman networks. The algorithm was independently derived by numerous researchers.^{[1][2][3]}

Algorithm [\[edit \]](#)

The training data for a recurrent neural network is an ordered sequence of k input-output pairs,



$\langle \mathbf{a}_0, \mathbf{y}_0 \rangle, \langle \mathbf{a}_1, \mathbf{y}_1 \rangle, \langle \mathbf{a}_2, \mathbf{y}_2 \rangle, \dots, \langle \mathbf{a}_{k-1}, \mathbf{y}_{k-1} \rangle$. An initial value must be specified for the hidden state $\mathbf{x}_0$. Typically, a vector of all zeros is used for this purpose.

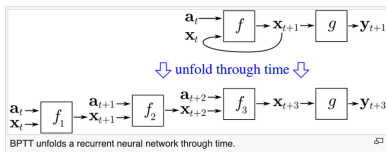
BPTT begins by unfolding a recurrent neural network in time. The unfolded network contains k inputs and outputs, but every copy of the network shares the same parameters. Then the [backpropagation](#) algorithm is used to find the gradient of the cost with respect to all the network parameters.

How does the Backpropagation through time work in RNN?

Backpropagation through time (BPTT) is a gradient-based technique for training certain types of recurrent neural networks. It can be used to train Elman networks. The algorithm was independently derived by numerous researchers.^{[1][2][3]}

Algorithm [\[edit \]](#)

The training data for a recurrent neural network is an ordered sequence of k input-output pairs,



$\langle a_0, y_0 \rangle, \langle a_1, y_1 \rangle, \langle a_2, y_2 \rangle, \dots, \langle a_{k-1}, y_{k-1} \rangle$. An initial value must be specified for the hidden state x_0 . Typically, a vector of all zeros is used for this purpose.

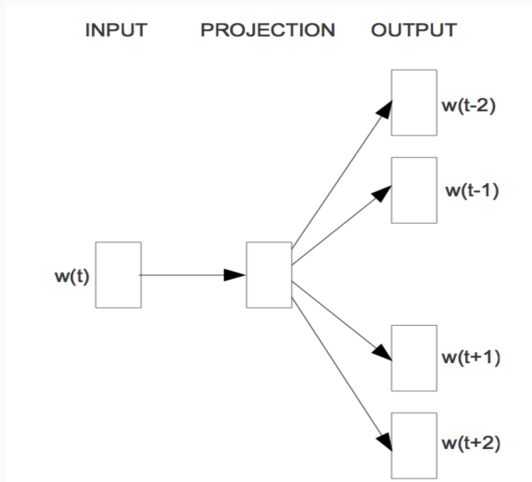
BPTT begins by unfolding a recurrent neural network in time. The unfolded network contains k inputs and outputs, but every copy of the network shares the same parameters. Then the [backpropagation](#) algorithm is used to find the gradient of the cost with respect to all the network parameters.

How does part-of-speech tagging work in NLP? ?

1. The task itself is to predict POS-tag.
2. rule-based
3. ML model to predict (e.g., hidden markov, naive bayes, LSTM and so one.

What is negative sampling when training the skip-gram model?

- Predict neighbors using target word



More on skip-gram

- Learn the probability $P(w_{t+j}|w_t)$: the probability to see w_{t+j} in target word w_t 's neighborhood
- Every word has two embeddings:
 - v_i serves as the role of target
 - u_i serves as the role of context
- Model probability as softmax:

$$P(o|c) = \frac{e^{u_o^T v_c}}{\sum_{w=1}^W e^{u_w^T v_c}}$$

- Learn embeddings to minimize the cross entropy loss

$$\min_{\{u_i\}, \{v_j\}} \sum_{(p,q) \in \text{positive pairs}} -\log\left(\frac{e^{u_p^T v_q}}{\sum_{r \neq p} e^{u_r^T v_q}}\right)$$

- Considering all $r \neq p$ is time consuming (size of vocabulary is large)
- Negative sampling:

$$\min_{\{u_i\}, \{v_j\}} \sum_{(p,q) \in \text{positive pairs}} -\log\left(\frac{e^{u_p^T v_q}}{\sum_{r \in D_{\text{negative}}} e^{u_r^T v_q}}\right)$$

where D_{negative} contains a subset of negative samples.

What is the term frequency-inverse document frequency (TF-IDF)? (document)

- Use the bag-of-words matrix or the normalized version (TF-IDF) for a dataset (denoted by D):

$$\text{tfidf}(\text{doc}, \text{word}, D) = \text{tf}(\text{doc}, \text{word}) \cdot \text{idf}(\text{word}, D)$$

- $\text{tf}(\text{doc}, \text{word})$: term frequency

(word count in the document)/(total number of terms in the document)

- $\text{idf}(\text{word}, \text{Dataset})$: inverse document frequency

$\log((\text{Number of documents})/(\text{Number of documents with this word}))$

	angeles	los	new	post	times	york
d1	0	0	1	0	1	1
d2	0	0	1	1	0	1
d3	1	1	0	0	1	0

tf-idf

	angeles	los	new	post	times	york
d1	0	0	0.584	0	0.584	0.584
d2	0	0	0.584	1.584	0	0.584
d3	1.584	1.584	0	0	0.584	0

What are the different ways of representing documents ?

🏢 Organization Card

① About org cards

SentenceTransformers 🤖 is a Python framework for state-of-the-art sentence, text and image embeddings.

Install the [Sentence Transformers](#) library.

```
pip install -U sentence-transformers
```

The usage is as simple as:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('paraphrase-MiniLM-L6-v2')

# Sentences we want to encode. Example:
sentence = ['This framework generates embeddings for each input sentence']

# Sentences are encoded by calling model.encode()
embedding = model.encode(sentence)
```

What are the different ways of representing documents ?

📄 Organization Card

① About org cards

SentenceTransformers 🤖 is a Python framework for state-of-the-art sentence, text and image embeddings.

Install the [Sentence Transformers](#) library.

```
pip install -U sentence-transformers
```

The usage is as simple as:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('paraphrase-MiniLM-L6-v2')

# Sentences we want to encode. Example:
sentence = ['This framework generates embeddings for each input sentence']

# Sentences are encoded by calling model.encode()
embedding = model.encode(sentence)
```

Question: What's the format you are expecting of the output ?

Define the Bag of N-grams model in NLP.

What are the differences between rule-based, statistical-based and neural-based approaches in NLP?

How to handle out-of-vocabulary (OOV) words in NLP?

What is the difference between a word-level and character-level language model?

What is co-reference resolution?

What is the Hidden Markov Model, and How it's helpful in NLP tasks?

What is a recurrent neural network (RNN)?

How does the Backpropagation through time work in RNN?

What are the limitations of a standard RNN?

What is a long short-term memory (LSTM) network?

What is the GRU model in NLP?

What is the sequence-to-sequence (Seq2Seq) model in NLP?

How does the attention mechanism help in NLP?

What is the Transformer model?

What is the role of the self-attention mechanism in Transformers?

What is the purpose of the multi-head attention mechanism in Transformers?

What are positional encodings in Transformers, and why are they necessary?

Describe the architecture of the Transformer model.

What is the difference between a generative and discriminative

model in NLP?

What is the BLEU score?

List out the popular NLP task and their corresponding evaluation metrics.

What is perplexity used for?

Explain how the Markov assumption affects the bi-gram model?

What are the most common word embedding methods? explain each briefly.

What are the first few steps that you will take before applying an NLP algorithm to a given corpus?

List a few types of linguistic ambiguities

What are stop words?

GPT vs BERT vs BART vs ELMO

GPT v1 vs v2 vs v3

What is masked language modelling?

What is beam search

Key takeaways

1. Major architectures: LSTM (state-model) and Transformer (attention)
2. Everything is a vector(embedding)
3. Every NLP task can be modeled as a ML task.
4. Current trend is using large model (in terms of training dataset and model size) as a backbone and make adjustment on top of it.